Manual Dropout in Neural Networks: A Reduced-Scale Reproduction and Ablation Study

Satyam Gupta
Roll No. 24CS2032
Integrated Dual Degree in Computer Science & Engineering and Artificial Intelligence
Rajiv Gandhi Institute of Petroleum Technology (RGIPT)
Jais, Amethi, Uttar Pradesh, India

Abstract—Dropout regularizes deep neural networks by stochastically omitting hidden units during training to suppress feature co-adaptation. In this study, we implement inverted dropout from first principles in PyTorch without built-in library modules. We conduct a fixed-budget rate ablation across eight drop rates ($d \in \{0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7\}$) across three seeds $(42, 43, 44)$ on a three-hidden-layer MLP ($784 \to 256 \to 256 \to 256 \to 10$). On MNIST, mean validation error selects $d = 0.4$, lowering test error from $4.320 \pm 0.295\%$ to $3.843 \pm 0.282\%$ (11.03% relative error reduction). On the Fashion-MNIST extension using the same protocol, validation error selects $d = 0.2$, reducing test error from $14.663 \pm 0.175\%$ to $14.347 \pm 0.253\%$ (2.16% relative reduction). Conversely, higher rates degrade performance under the fixed 20-epoch budget. Under our reduced-scale experimental constraints, dropout provides measurable regularization, with optimal rates depending on dataset characteristics and requiring validation-based selection.

Index Terms—Dropout, regularization, neural networks, ablation study, model selection, MNIST, Fashion-MNIST.

I. Introduction

Deep neural networks possess remarkable expressive capacity, enabling them to approximate complex non-linear functions. However, when parameter capacity is large relative to the volume of training examples, networks readily memorize training noise and overfit, yielding high generalization error on unseen test data [1].

To mitigate overfitting, ensemble methods combine predictions from multiple independently trained models. While effective, training and maintaining large ensembles of deep neural networks is computationally prohibitive. In their seminal work, Srivastava et al. [1] introduced dropout as a computationally efficient approximation to model ensembling. By stochastically dropping units during each training iteration, dropout trains an exponential family of thinned subnetworks that share weights. At evaluation time, deterministic forward propagation can be interpreted as approximate model averaging over these thinned networks, rather than an exact arithmetic ensemble equivalence.

The objective of this assignment is to implement dropout from first principles using Python and PyTorch without relying on built-in primitives such as torch.nn.Dropout or torch.nn.functional.dropout. We evaluate this manual implementation through a controlled drop-rate ablation across eight uniform rates ($d \in \{0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7\}$) using three fixed initialization seeds $(42, 43, 44)$. We investigate both the primary MNIST handwritten digit benchmark [2] and an identical-protocol extension to Fashion-MNIST [3].

We emphasize that this project is an educational, reduced-scale reproduction rather than an attempt to replicate the exact state-of-the-art numerical accuracies of the original 2014 publication. We employ a fixed parameter width, a sub-sampled training set of 10,000 examples, and a constrained budget of 20 training epochs. Crucially, we adhere to strict scientific evaluation: hyperparameters and drop rates are chosen exclusively on validation data, leaving the official test set for unbiased descriptive evaluation. Our empirical findings demonstrate that while dropout yields noticeable error reductions at moderate rates, high dropout rates degrade generalization under fixed compute budgets, confirming that regularization strength must be calibrated carefully.

II. Dropout Formulation and Architecture

A. Mathematical Formulation

Let $h \in \mathbb{R}^D$ denote the post-activation vector of a hidden neural network layer. We define the drop probability as $d \in [0, 1)$ and the corresponding retention or keep probability as:

$$q = 1 - d, \quad q \in (0, 1]. \tag{1}$$

During training, a random binary mask vector $m \in \{0, 1\}^D$ is sampled independently across units from a Bernoulli distribution parameterized by q:

$$m_i \sim \text{Bernoulli}(q), \quad \mathbb{P}(m_i = 1) = q, \quad \mathbb{P}(m_i = 0) = d. \tag{2}$$

In modern deep learning implementations, inverted dropout is the standard formulation. Rather than scaling activations at test time, inverted dropout rescales the unmasked activations during training by the reciprocal of the keep probability, $1/q$:

$$\tilde{h}_i^{(\text{train})} = \frac{m_i \cdot h_i}{q}. \tag{3}$$

During evaluation (inference), the dropout operator is entirely bypassed:

$$\tilde{h}_i^{(\text{eval})} = h_i. \tag{4}$$

B. Expectation Derivation

The primary mathematical justification for the $1/q$ scaling factor is preservation of the expected activation magnitude across training and evaluation phases. For any arbitrary unit activation h_i, the conditional expectation of $\tilde{h}_i^{(\text{train})}$ given h_i is derived as follows:

$$\mathbb{E}\left[\tilde{h}_i^{(\text{train})} \mid h_i\right] = \mathbb{E}\left[\frac{m_i \cdot h_i}{q} \,\middle|\, h_i\right] \tag{5}$$

$$= \frac{h_i}{q}\,\mathbb{E}\left[m_i\right] \tag{6}$$

$$= \frac{h_i}{q} \cdot (1 \cdot \mathbb{P}(m_i = 1) + 0 \cdot \mathbb{P}(m_i = 0)) \tag{7}$$

$$= \frac{h_i}{q} \cdot q \tag{8}$$

$$= h_i. \tag{9}$$

Inverted dropout preserves the first moment of each activation during training ($\mathbb{E}\left[\tilde{h}_i^{(\text{train})} \mid h_i\right] = \tilde{h}_i^{(\text{eval})} = h_i$) and therefore requires no additional activation scaling at evaluation. Downstream weights do not require post-hoc rescaling at inference time, ensuring that evaluation mode remains deterministic. Furthermore, dropout can be interpreted as approximate model averaging over many thinned networks, not as an exact arithmetic ensemble equivalence.

C. Inverted Dropout vs. Original Paper Formulation

In the original presentation by Srivastava et al. [1], dropout was formulated using standard test-time scaling: during training, activations were unscaled ($\tilde{h}_i = m_i \cdot h_i$), and at test time, outgoing weights or activations were multiplied by the keep probability ($\tilde{h}_i = q \cdot h_i$). Reference materials on the author's project page [4] and the historical Deepnet repository [5] document this classical convention.

Each convention matches its training first moment to its own evaluation rule: classical dropout gives qh_i at evaluation, whereas inverted dropout gives h_i. They do not produce identical outputs for unchanged weights. Inverted dropout is widely preferred in modern production frameworks (such as PyTorch [6]) because it restricts stochasticity and scaling logic entirely to the training graph; at inference time, evaluation is an unscaled identity forward pass without extra arithmetic operations or memory allocations.

D. Network Architecture

All experiments employ a Multilayer Perceptron (MLP) consisting of three fully connected hidden layers:

$$\mathbf{x} \in \mathbb{R}^{784} \xrightarrow{W_1,b_1} \mathbb{R}^{256} \xrightarrow{W_2,b_2} \mathbb{R}^{256} \xrightarrow{W_3,b_3} \mathbb{R}^{256} \xrightarrow{W_4,b_4} \mathbf{y} \in \mathbb{R}^{10}. \tag{10}$$

Each linear hidden layer is followed by a Rectified Linear Unit (ReLU) activation function:

$$\sigma(z) = \max(0, z). \tag{11}$$

Our custom ManualDropout layer is inserted immediately after each hidden ReLU activation. In strict accordance with the project specification:

- No dropout is applied to the raw input layer ($d_{\text{in}} = 0$).
- No dropout is applied to the final classification logits.
- Built-in dropout routines (torch.nn.Dropout) are strictly forbidden.
- The total trainable parameter count is exactly:

$$(784 \times 256 + 256) + 2 \times (256 \times 256 + 256) + (256 \times 10 + 10) = 335{,}114. \tag{12}$$

III. Experimental Setup

A. Protocol Specification (dropout-24cs2032-v1)

We freeze the experimental configuration under protocol identifier dropout-24cs2032-v1. Key experimental parameters are detailed in Table I.

TABLE I
Frozen Experimental Protocol Parameters

Parameter	Specification
Architecture	$784 \to 256 \to 256 \to 256 \to 10$ MLP
Hidden Activation	Rectified Linear Unit (ReLU)
Dropout Placement	After hidden ReLUs (Layers 1, 2, 3)
Drop Rates (d)	$0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7$
Keep Rates (q)	$1.0, 0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3$
Initialization Seeds	$42, 43, 44$ (3 seeds)
Split Seed	2026
Training Set Size	10,000 examples (disjoint subset)
Validation Set Size	5,000 examples (disjoint subset)
Test Set Size	10,000 examples (official test split)
Training Epochs	20 fixed epochs
Mini-Batch Size	128
Optimizer	Adam ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$)
Learning Rate	0.001 (fixed, no decay schedule)
Weight Decay	0.0 (no L_2 regularization)
Other Regularizers	None (no batch norm, no max-norm)
Data Preprocessing	Division by 255.0 (float32 scaling)
Compute Backend	CPU (Apple Silicon arm64, native)

B. Data Partitioning and Fair Initialization

The official torchvision datasets [7], [8] provide 60,000 training and 10,000 test images. To operate within a standardized educational compute budget, we fix split seed 2026 to partition the 60,000 training images into 10,000 training examples, 5,000 validation examples, and 45,000 unused images. The official test set (10,000 images) was evaluated each epoch for descriptive learning curves. It was not used for gradient updates, dropout-rate selection, checkpoint selection, or training-budget decisions.

To ensure strict experimental fairness across drop rates, we enforce deterministic initialization: for any given random seed ($s \in \{42, 43, 44\}$), the model weights across all eight drop rates ($d \in [0.0, 0.7]$) are verified to be bitwise identical at step 0 before optimization begins.

C. Model Selection and Evaluation Policy

Model selection is governed strictly by minimum validation error:

$$d^* = \arg\min_d \ \overline{\text{Error}}_{\text{val}}(d), \tag{13}$$

where $\overline{\text{Error}}_{\text{val}}(d)$ is the mean classification error across the three seeds at epoch 20. Ties are resolved by selecting the lower drop rate.

In strict adherence to sound scientific methodology, test set performance is never consulted during rate selection, checkpoint picking, or hyperparameter adjustment. All reported test errors are unbiased descriptive metrics evaluated at the fixed termination epoch ($E = 20$). Statistical dispersion is reported using the sample standard deviation ($ddof = 1$).

D. Hardware and Execution Environment

Experiments were executed on macOS 26.6.2 (Darwin arm64) using Python 3.13.9, PyTorch 2.14.1, torchvision 0.29.1, and NumPy 2.5.3. Preliminary benchmarks revealed that CPU execution required 0.785 seconds per 2-epoch run compared to 4.398 seconds on Apple Metal Performance Shaders (MPS), representing an efficiency advantage of over 5.5× for CPU on this small model. This performance difference may reflect accelerator launch and host-device transfer overhead for this small workload (335,114 parameters). Consequently, CPU was locked as the deterministic compute backend.

IV. Experimental Results

A. MNIST Main Reproduction

The complete sweep of 24 independent runs (8 rates × 3 seeds) was executed to fixed epoch 20. Table II details the training, validation, and test error rates across all evaluated drop rates.

TABLE II
MNIST Results at Epoch 20 (Mean ± SD over Seeds 42, 43, 44)

Rate (d)	Train Error (%)	Val Error (%)	Test Error (%)
0.0 (Baseline)	0.203 ± 0.200	4.260 ± 0.533	4.320 ± 0.295
0.1	0.090 ± 0.044	4.180 ± 0.314	4.050 ± 0.278
0.2	0.110 ± 0.000	3.893 ± 0.099	3.900 ± 0.239
0.3	0.240 ± 0.080	3.753 ± 0.061	3.820 ± 0.118
0.4 (Selected)	$\mathbf{0.323 \pm 0.047}$	$\mathbf{3.747 \pm 0.130}$	$\mathbf{3.843 \pm 0.282}$
0.5 (Reference)	0.700 ± 0.044	3.840 ± 0.140	3.883 ± 0.087
0.6	1.273 ± 0.057	4.407 ± 0.219	4.380 ± 0.104
0.7	2.513 ± 0.153	5.133 ± 0.090	5.110 ± 0.147

Baseline Performance: The unregularized baseline ($d = 0.0$) achieves a training error of $0.203 \pm 0.200\%$, validation error of $4.260 \pm 0.533\%$, and test error of $4.320 \pm 0.295\%$, representing a substantial generalization gap of 4.117 ± 0.101 percentage points (pp).

Validation-Selected Model: The lowest mean validation error is attained at drop rate $d = 0.4$ ($3.747 \pm 0.130\%$). At this rate, the model achieves a test error of $3.843 \pm 0.282\%$ (accuracy: 96.157%). Compared to the unregularized baseline, $d = 0.4$ yields an absolute reduction in test error of 0.477 pp, corresponding to a relative test error reduction of:

$$\frac{4.320 - 3.843}{4.320} \times 100\% = 11.03\%. \tag{14}$$

Furthermore, the generalization gap narrows from 4.117 pp to 3.520 ± 0.295 pp.

Test Set Nuance: Descriptively, the lowest numerical test error was observed at $d = 0.3$ ($3.820 \pm 0.118\%$). However, because model selection is strictly confined to validation data, $d = 0.4$ remains the sole authorized selected model. The difference between test error at $d = 0.3$ (3.820%) and $d = 0.4$ (3.843%) is minor (0.023 pp) and falls well within the sample standard deviation (0.282%).

Negative Findings: When the drop rate is increased to $d = 0.6$ and $d = 0.7$, test performance deteriorates to $4.380 \pm 0.104\%$ and $5.110 \pm 0.147\%$ respectively, both performing markedly worse than the unregularized baseline.

B. Fashion-MNIST Extension

To evaluate whether these regularizing properties generalize beyond digit recognition, we executed the identical 24-run protocol on Fashion-MNIST [3] as an extension using the same experimental protocol. Table III summarizes the results.

TABLE III
Fashion-MNIST Results at Epoch 20 (Mean ± SD over Seeds 42, 43, 44)

Rate (d)	Train Error (%)	Val Error (%)	Test Error (%)
0.0 (Baseline)	7.130 ± 0.177	13.127 ± 0.247	14.663 ± 0.175
0.1	7.587 ± 0.471	13.113 ± 0.042	14.220 ± 0.105
0.2 (Selected)	$\mathbf{8.583 \pm 0.414}$	$\mathbf{13.087 \pm 0.408}$	$\mathbf{14.347 \pm 0.253}$
0.3	9.393 ± 0.067	13.267 ± 0.244	14.567 ± 0.396
0.4	10.060 ± 0.584	13.573 ± 0.494	14.647 ± 0.330
0.5 (Reference)	10.990 ± 0.366	13.747 ± 0.200	15.133 ± 0.325
0.6	12.273 ± 0.323	14.380 ± 0.053	15.730 ± 0.373
0.7	14.543 ± 0.468	15.673 ± 0.380	17.453 ± 0.376

Baseline Performance: The unregularized baseline ($d = 0.0$) produces a training error of $7.130 \pm 0.177\%$, validation error of $13.127 \pm 0.247\%$, and test error of $14.663 \pm 0.175\%$ with a generalization gap of 7.533 ± 0.351 pp.

Validation-Selected Model vs. Descriptive Test-Best Rate: Validation error selects drop rate $d = 0.2$ ($q = 0.8$) based on minimum validation error ($13.087 \pm 0.408\%$), achieving test error $14.347 \pm 0.253\%$ (accuracy: 85.653%, +0.317 pp improvement over baseline, gap: 5.763 ± 0.165 pp). Descriptively, lowest numerical mean test error was observed at $d = 0.1$ ($q = 0.9$, $14.220 \pm 0.105\%$, val error $13.113 \pm 0.042\%$). Because selection relies strictly on validation error, $d = 0.2$ ($q = 0.8$) remains the sole selected model; $d = 0.1$ must not be presented as the selected configuration.

Critical Negative Finding: Notably, the standard paper reference rate of $d = 0.5$ yields a test error of

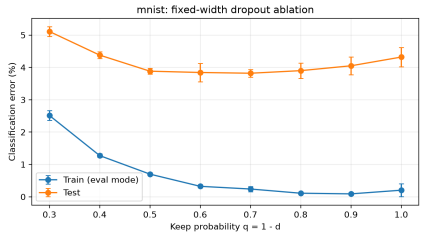


Fig. 1. Reduced-scale fixed-width dropout rate ablation on MNIST inspired by Srivastava et al. [1], Fig. 9(a). Data points depict measured experimental results (mean ± sample SD across three seeds) plotted against keep probability $q = 1 - d$.

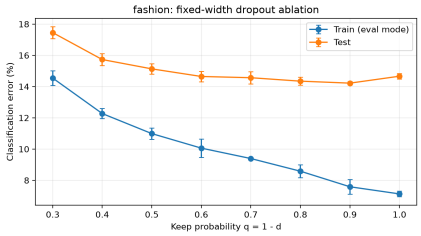


Fig. 2. Keep-probability ablation curve for Fashion-MNIST extension under protocol dropout-24cs2032-v1. Error increases monotonically for retention probabilities below $q = 0.8$ ($d > 0.2$).

$15.133 \pm 0.325\%$, which is 0.470 pp worse than the no-dropout baseline. At $d = 0.7$, test error degrades sharply to $17.453 \pm 0.376\%$. This highlights that applying a canonical drop rate of $d = 0.5$ without validation tuning can actively harm model accuracy on more complex visual benchmarks.

C. Learning Dynamics and Overfitting

Figure 3 illustrates the epoch-wise training error, test error, and validation loss trajectories on MNIST for the baseline ($d = 0.0$) and reference dropout ($d = 0.5$).

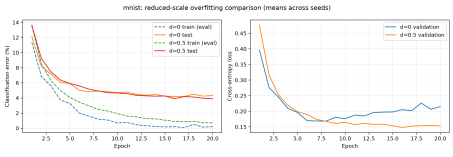


Fig. 3. Epoch-wise learning dynamics on MNIST comparing unregularized training ($d = 0.0$) against dropout ($d = 0.5$) under our reduced-scale protocol (10,000 training examples, 20 epochs). While the unregularized model drives training error toward zero and exhibits rising validation loss (overfitting), dropout regularizes the representation, maintaining stable validation loss and lower test error.

In the baseline model ($d = 0.0$), training error rapidly declines toward 0.2%, while validation loss diverges after epoch 6, evidencing severe memorization. In contrast, with dropout enabled ($d = 0.5$), training error declines more gradually, and validation loss stabilizes at a lower plateau, illustrating effective regularization of network capacity.

D. Keep-Probability Ablation Curves

Figure 1 and Figure 2 present the relationships between test error and keep probability $q = 1 - d$ for MNIST and Fashion-MNIST respectively.

The MNIST curve (Fig. 1) displays a distinct U-shaped profile. When $q = 1.0$ (no dropout), overfitting leads to elevated test error (4.320%). As q decreases into the intermediate regime ($q \in [0.6, 0.9]$, corresponding to $d \in [0.1, 0.4]$), test error reaches an optimal trough between 3.820% and 3.900%. However, when retention falls to $q \leq 0.4$ ($d \geq 0.6$), test error rises steeply to 5.110%.

On Fashion-MNIST (Fig. 2), validation selected $q = 0.8$ ($d = 0.2$). The descriptive lowest mean test error occurred at $q = 0.9$ ($d = 0.1$). As retention probability decreases below $q = 0.5$ ($d > 0.5$), performance degrades rapidly.

V. Discussion and Comparison with Srivastava et al. (2014)

A. Comparison with Original Findings

In Table 2 of Srivastava et al. [1], the authors report a test error of 1.25% for a standard feedforward neural network with dropout and ReLU activations on MNIST (3×1024 hidden units, 60,000 training examples, SGD with momentum). In our reduced-scale reproduction, our baseline achieves $4.320 \pm 0.295\%$ and our validation-selected model achieves $3.843 \pm 0.282\%$.

Our reduced-scale experiment is not an apples-to-apples comparison with the original paper. As detailed below, these results reflect fundamentally different experimental regimes:

1) Network Capacity: Srivastava et al. evaluated networks with 3×1024 units (and up to 8,192 units in wider experiments), whereas our model utilizes 256 units per hidden layer (335,114 total parameters).
2) Dataset Scale: The original paper utilized all 60,000 available training examples, compared to our subsampled set of 10,000 training examples.
3) Training Budget and Optimization: The original work trained networks for hundreds of epochs with learning-rate schedules and momentum. The 1.25% result in Table 2 corresponds specifically to their 3×1024 ReLU dropout architecture; max-norm regularized variants ($\|W\|_2 \leq c$) represent separate experimental configurations in the paper. By contrast, our experiment operates without weight constraints and employs Adam for a constrained budget of 20 epochs.
4) Regularization Formulation: We utilize modern inverted dropout during training rather than test-time weight rescaling.

Separately, Figure 9(a) of Srivastava et al. investigates the effect of varying retention probability on a fixed-architecture network. While our rate sweep is inspired by this study, ours is a reduced-scale ablation rather than an exact replication. Across our experiments, generalization

error demonstrates a qualitatively similar non-monotonic response to retention probability: intermediate keep probabilities provide effective regularization, whereas higher train and test errors at high drop rates are consistent with stronger underfitting under the fixed 20-epoch budget. The Fashion-MNIST benchmark has no counterpart in the 2014 paper and is evaluated strictly as an extension using the same experimental protocol.

B. Differential Regularization Behavior

A compelling finding of this study is the marked difference in optimal drop rate between MNIST ($d^* = 0.4$) and Fashion-MNIST ($d^* = 0.2$). MNIST digits consist of clean, high-contrast, stroke-based patterns with substantial redundant pixel information, making the representations highly amenable to aggressive unit deactivation.

Conversely, Fashion-MNIST exhibits complex, subtle textures and fine-grained visual distinctions between apparel categories. High drop rates produced higher train and test error under the fixed 20-epoch budget, consistent with stronger underfitting for $d \geq 0.5$.

VI. Failure Analysis and Limitations

A. Empirical Failure Case Analysis

To inspect the nature of model errors, we examined misclassified test examples from the validation-selected models across both datasets. Figure 4 displays representative test failure cases.



Fig. 4. Representative misclassified test instances on MNIST ($d = 0.4$, left) and Fashion-MNIST ($d = 0.2$, right) showing true labels, model predictions, and softmax confidences.

The inspected examples on MNIST (Fig. 4, left) include ambiguous handwriting: e.g., true label 2 classified as 3 (confidence 0.52) and true label 9 classified as 8 (confidence 0.91). In these specific instances, stroke patterns visually resemble the predicted class, though this observation does not establish an isolated causal mechanism or describe the entire error distribution.

On Fashion-MNIST (Fig. 4, right), the inspected examples include pairs from semantically adjacent categories: true Coat classified as Pullover (confidence 0.71), and true Ankle Boot classified as Sneaker (confidence 0.99). At 28×28 grayscale resolution, silhouettes of coats and pullovers or boots and sneakers share visual shape resemblances that may contribute to confusion under low resolution and small budgets, but the displayed examples do not establish category-wide concentration or identify individual misclassification causes. Fully connected architectures lack spatial convolutional inductive bias, making subtle texture distinctions challenging to resolve.

B. Methodological Limitations

We identify the following methodological constraints:

1) Seed Sample Size: All results are evaluated over three random seeds $(42, 43, 44)$. While reporting sample standard deviation with $ddof = 1$ reflects variance, this sample size is insufficient for formal statistical significance testing.
2) Fixed Training Budget: All models were trained for exactly 20 epochs. High dropout rates inherently slow down convergence; allocating additional epochs might partially recover performance for $d \geq 0.5$.
3) Subsampled Training Split: Using 10,000 training examples magnifies regularizer sensitivity relative to the full 60,000 dataset.
4) Cloud Portability and Numerical Equivalence: A separate complete MNIST CPU run was executed in Google Colab using the same source SHA256 and saved split indices, completing all 24 runs. The Colab environment selected $d = 0.3$, while the primary Mac CPU experiment selected $d = 0.4$. The primary reported results remain the Mac CPU experiment; exact cross-platform numerical equivalence is not claimed, with differing architecture backends and BLAS library routines serving as plausible contributing factors.

VII. Conclusion

This study presented a complete, from-scratch implementation of inverted dropout in PyTorch, accompanied by an empirical ablation study across eight drop rates on MNIST and Fashion-MNIST. By strictly adhering to validation-based model selection, we demonstrated that moderate dropout rates reduced mean test error relative to the unregularized baseline in our experimental setup (11.03% relative reduction on MNIST at $d = 0.4$; 2.16% relative reduction on Fashion-MNIST at $d = 0.2$).

Crucially, our experiments also revealed negative results: canonical rates such as $d = 0.5$ underperformed the baseline on Fashion-MNIST, and high drop rates ($d \geq 0.6$) degraded accuracy across both benchmarks. These findings corroborate the core regularization hypothesis of Srivastava et al. while confirming that dropout is not universally beneficial without systematic validation calibration.

Assistance and Provenance Statement

The initial core implementation was supplied in a ChatGPT-generated project guide. ChatGPT and Antigravity also supported review, execution workflow, and documentation. All experimental results, figures, tables, and numerical metrics presented in this paper originate entirely from recorded real-data runs executed in this project repository and independently audited from persistent artifacts.

References

[1] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, no. 56, pp. 1929–1958, 2014.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[3] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," arXiv preprint arXiv:1708.07747, 2017.

[4] N. Srivastava, "Dropout: A simple way to prevent neural networks from overfitting (project page)," https://www.cs.toronto.edu/~nitish/dropout/, 2014, accessed: 2026-10-05.

[5] ——, "deepnet: Implementation of deep learning models in python," https://github.com/nitishsrivastava/deepnet, 2014, consulted for historical reference only; not a runtime dependency.

[6] PyTorch Core Team, "Dropout module specification and inverted scaling formula," https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout.html, 2026, accessed: 2026-10-05.

[7] TorchVision Maintainers, "Torchvision datasets: Mnist specification," https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.MNIST.html, 2026, accessed: 2026-10-05.

[8] ——, "Torchvision datasets: Fashion-mnist specification," https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.FashionMNIST.html, 2026, accessed: 2026-10-05.